MLM PLATFORM

Django Backend — Complete Cursor AI Prompt

Python · Django 5.x · DRF	MySQL · Redis · Celery	Docker + Nginx	DS Rules 2021 Compliant

Prepared for: Internal Development Team

Architecture: Monolithic Django REST API · JWT Auth · Razorpay · Binary MLM

0. Context & Objective

Build a complete, production-ready Django REST API backend for a legally compliant, product-based Binary MLM platform. The platform sells a ₹200 digital e-book course (₹241.72 all-inclusive with GST + Razorpay charges). All commissions and bonuses are tied strictly to product sales — never to recruitment headcount. The backend must serve three clients:

- Flutter Mobile App — primary member interface
- React / Next.js Website — public marketing + web member portal
- Internal React Admin Panel — back-office operations

1. Project Structure

```
mlm backend/
├── config/
│   ├── settings/  (base.py · development.py · production.py)
│   ├── urls.py
│   ├── wsgi.py
│   └── celery.py
├── apps/
│   ├── authentication/    # OTP login, JWT, KYC
│   ├── users/             # Member profile, referral code, placement
│   ├── mlm tree/          # Binary tree logic, node management
│   ├── commissions/       # Commission engine, milestone bonuses
│   ├── wallet/            # Wallet, withdrawal bands, TDS
│   ├── sponsor slots/     # Sponsored Enrollment Slot System
│   ├── courses/           # E-book management, enrollment, access
│   ├── payments/          # Razorpay integration, GST invoicing
│   ├── admin_panel/       # Admin views, reporting, config
│   ├── notifications/     # SMS, push, email notifications
│   └── audit/             # Audit log for all critical events
├── docker/
│   ├── Dockerfile
│   ├── docker-compose.yml
│   └── nginx/default.conf
├── requirements/
│   ├── base.txt
│   └── production.txt
└── manage.py
```

2. Docker & Infrastructure

Docker Compose Services

Service	Purpose	Port
web	Django + Gunicorn application server	8000
nginx	Reverse proxy, SSL termination	80 / 443
db	MySQL 8.x with persistent volume	3306
redis	Celery broker + caching layer	6379
celery	Async task worker (same image as web)	—
celery-beat	Scheduled task runner (cron jobs)	—

Environment Variables (.env)

```
SECRET_KEY, DEBUG, ALLOWED_HOSTS
DB_HOST, DB_NAME, DB_USER, DB_PASSWORD, DB_PORT
REDIS_URL
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET
RAZORPAY_PAYOUT_KEY_ID, RAZORPAY_PAYOUT_KEY_SECRET
SMS_PROVIDER_API_KEY   # MSG91 or Fast2SMS
AADHAAR_OTP_API_KEY
FRONTEND_BASE_URL
GST_NUMBER, PAN_NUMBER, COMPANY_NAME
JWT_ACCESS_TOKEN_LIFETIME_MINUTES=60
JWT_REFRESH_TOKEN_LIFETIME_DAYS=7
```

3. Core Business Constants

All values below must be stored in SystemConfig model (admin-editable without redeployment).

Constant	Value	Description
PRODUCT_BASE_PRICE	₹200	Course base price ex-GST
GST_RATE	18%	SAC Code 9992
RAZORPAY_CHARGE	₹5.72	Payment gateway fee
PRODUCT_TOTAL	₹241.72	All-inclusive checkout price
DIRECT_COMMISSION	₹30	Per direct referral sale
UPLINE_COMMISSION	₹10	Per binary upline (L2, L3, L4)
MAX_COMMISSION_PER_ORDER	₹60	Total distributed per order (30% of ₹200)
EARNING_CAP	₹22,200	Lifetime maximum per account
SPONSOR_SLOT_COUNT	5	Codes per slot batch
SPONSOR_SLOT_EXPIRY_DAYS	20	Code validity window
REFUND_WINDOW_DAYS	7	DS Rules 2021, Rule 5(5)
TDS_194H_RATE	2%	Commission TDS (above ₹20K/FY)
TDS_194R_RATE	10%	Milestone bonus TDS (above ₹20K/FY)
TDS_TRIGGER_CASH	₹20,000	FY threshold for TDS activation
PAN_MISSING_TDS_RATE	20%	TDS rate if PAN not provided

Milestone Bonus Table

Referrals	Sales Value	Bonus %	Bonus Paid	Cumul. Bonus	Member Total (comm+bonus+tree)
10	₹2,000	15%	₹300	₹300	₹600
25	₹5,000	12%	₹600	₹900	₹1,650
50	₹10,000	10%	₹1,000	₹1,900	₹3,700
75	₹15,000	9%	₹1,350	₹3,250	₹5,500
100	₹20,000	8%	₹1,600	₹4,850	₹8,150

Withdrawal Band Table

Band	Earnings Range	Type	Value	TDS?
1	₹200 → ₹4,000	CASH	₹3,800	No
2	₹4,000 → ₹5,000	SLOT (5 codes)	₹1,000	No (not cash)
3	₹5,000 → ₹9,000	CASH	₹4,000	No
4	₹9,000 → ₹10,000	SLOT (5 codes)	₹1,000	No (not cash)
5	₹10,000 → ₹14,000	CASH	₹4,000	YES — 2% above ₹20K FY
6	₹14,000 → ₹15,000	SLOT (5 codes)	₹1,000	No (not cash)
7	₹15,000 → ₹19,000	CASH	₹4,000	YES — 2%
8	₹19,000 → ₹20,000	SLOT (5 codes)	₹1,000	No (not cash)
9	₹20,000 → ₹22,200	CASH	₹2,200	YES — 2%
TOTAL	₹22,200 full cycle	4× Cash + 4× Slots	₹18,000 cash + ₹4,000 slots	~₹164 TDS total

4. Database Models

4.1 OTPRecord (apps/authentication/models.py)

```
class OTPRecord(Model):
    phone           = CharField(max_length=15, null=True)
    email           = EmailField(null=True)
    otp_code        = CharField(max_length=6)
    purpose         = CharField(choices=['REGISTER','LOGIN','KYC','ADMIN LOGIN'])
    is_used         = BooleanField(default=False)
    attempts        = IntegerField(default=0)    # max 5
    expires_at      = DateTimeField()            # +10 minutes
    ip_address      = GenericIPAddressField(null=True)
    device_fingerprint = CharField(null=True)
```

4.2 User (apps/users/models.py)

```
class User(AbstractBaseUser, PermissionsMixin):
    # Identity
    phone           = CharField(max_length=15, unique=True, null=True)
    email           = EmailField(unique=True, null=True)
    full_name       = CharField(max_length=255)
    # KYC
    pan_number      = CharField(max_length=10, null=True)
    aadhaar_number  = CharField(max_length=12, null=True)  # stored masked
    kyc_status      = CharField(choices=['PENDING','VERIFIED','REJECTED'])
    # Roles
    is_member       = BooleanField(default=False)  # purchased the course
    role            = CharField(choices=['SUPER_ADMIN','FINANCE','SUPPORT','MEMBER'])
    # MLM Identity
```

```
    member_id       = CharField(unique=True)   # e.g. MLM000001
    referral_code   = CharField(unique=True)   # 8-char alphanumeric
    referral_link   = URLField()               # platform.com/join?ref=CODE
    sponsor         = ForeignKey('self', null=True, related_name='direct_referrals')
    # Account lifecycle
    account_status  = CharField(choices=['ACTIVE','CAPPED','SUSPENDED','DEACTIVATED'])
    # Bank/UPI for payouts
    bank_account_number = CharField(null=True)
    bank_ifsc       = CharField(null=True)
    upi_id          = CharField(null=True)
    payout_preference = CharField(choices=['BANK','UPI'], default='UPI')
```

4.3 BinaryNode (apps/mlm_tree/models.py)

```
class BinaryNode(Model):
    user        = OneToOneField(User, related_name='binary_node')
    parent      = ForeignKey('self', null=True, related_name='children')
    position    = CharField(choices=['LEFT','RIGHT'], null=True)
    level       = IntegerField(default=1)   # depth (root=1)
    left_child  = OneToOneField('self', null=True)
    right_child = OneToOneField('self', null=True)
    # Denormalized upline chain for fast commission calculation
    upline_l1   = ForeignKey(User, null=True, related_name='downline_l1')
    upline_l2   = ForeignKey(User, null=True, related_name='downline_l2')
    upline_l3   = ForeignKey(User, null=True, related_name='downline_l3')
```

4.4 CommissionLedger & MilestoneRecord (apps/commissions/models.py)

```
class CommissionLedger(Model):
    recipient       = ForeignKey(User, related_name='commissions_received')
    source_user     = ForeignKey(User)   # who bought the course
    order           = ForeignKey('payments.Order')
    commission_type =
CharField(choices=['DIRECT','UPLINE_L2','UPLINE_L3','UPLINE_L4','MILESTONE'])
    amount          = DecimalField(max_digits=10, decimal_places=2)
    tds_deducted    = DecimalField(default=0)
    net_amount      = DecimalField()
    status          = CharField(choices=['PENDING','CREDITED','REVERSED','HELD'])

class MilestoneRecord(Model):
    user                = ForeignKey(User)
    milestone_referrals = IntegerField()   # 10, 25, 50, 75, or 100
    bonus_amount        = DecimalField()
    tds_deducted        = DecimalField(default=0)
    net_bonus           = DecimalField()
    status              = CharField(choices=['PENDING','CREDITED'])
```

4.5 Wallet & WithdrawalRequest (apps/wallet/models.py)

```
class Wallet(Model):
    user                = OneToOneField(User)
    cash_balance        = DecimalField(default=0)
    total_earned        = DecimalField(default=0)     # all-time
    total_withdrawn     = DecimalField(default=0)
    total_tds_deducted  = DecimalField(default=0)
    current_band        = IntegerField(default=1)     # 1-9
    band_cash_withdrawn_fy = DecimalField(default=0)  # FY tracking for TDS

class WithdrawalRequest(Model):
    user            = ForeignKey(User)
    band            = IntegerField()                # band number 1-9
    amount_requested = DecimalField()
    tds_amount      = DecimalField(default=0)
    net_payable     = DecimalField()
    tds_section     = CharField(null=True)          # '194H' or '194R'
    status          =
CharField(choices=['PENDING','APPROVED','PROCESSING','PAID','REJECTED','FAILED'])
    razorpay_payout_id = CharField(null=True)
```

```
    utr_number      = CharField(null=True)
```

4.6 SponsorSlot Models (apps/sponsor_slots/models.py)

```
class SponsorSlotBatch(Model):
    issued_to       = ForeignKey(User)
    band_number     = IntegerField()        # which band triggered this
    total_codes     = IntegerField(default=5)
    codes_redeemed  = IntegerField(default=0)
    codes_expired   = IntegerField(default=0)
    expires_at      = DateTimeField()       # issued at + 20 days

class SponsorSlotCode(Model):
    batch           = ForeignKey(SponsorSlotBatch)
    issued_to       = ForeignKey(User)
    code            = CharField(unique=True) # e.g. SPONSOR-A3X7K2
    status          = CharField(choices=['ACTIVE','REDEEMED','EXPIRED','SHARED'])
    shared_via      = CharField(choices=['WHATSAPP','SMS','EMAIL','COPY'], null=True)
    redeemed_by     = ForeignKey(User, null=True)
    redeemed_order  = ForeignKey('payments.Order', null=True)
    expires_at      = DateTimeField()
    # Abuse detection
    unique_ips_attempted = JSONField(default=list)
    is_flagged      = BooleanField(default=False)
```

4.7 EBook & Enrollment (apps/courses/models.py)

```
class EBook(Model):
    title       = CharField(max_length=255)
    slug        = SlugField(unique=True)
    category    = CharField()   # Finance, Health, Business...
    file_url    = URLField()    # secure Azure Blob CDN URL
    is_active   = BooleanField(default=True)
    is_primary  = BooleanField(default=False)  # the ₹200 course product

class Enrollment(Model):
    user        = ForeignKey(User)
    ebook       = ForeignKey(EBook)
    order       = ForeignKey('payments.Order')
    is_retail   = BooleanField(default=False)  # retail vs MLM member
    download_count = IntegerField(default=0)
```

4.8 Order & GSTInvoice (apps/payments/models.py)

```
class Order(Model):
    user                = ForeignKey(User)
    order_number        = CharField(unique=True)   # ORD-20260408-XXXXX
    base_price          = DecimalField(default=200)
    gst_amount          = DecimalField(default=36)
    gateway_charge      = DecimalField(default=5.72)
    total_amount        = DecimalField(default=241.72)
    discount_amount     = DecimalField(default=0)
    amount_paid         = DecimalField()
    sponsor_code_used   = ForeignKey('sponsor_slots.SponsorSlotCode', null=True)
    is_sponsor_slot_redemption = BooleanField(default=False)
    razorpay_order_id   = CharField(null=True)
    razorpay_payment_id = CharField(null=True)
    status              = CharField(choices=['CREATED','PAID','FAILED','REFUNDED'])
    gst_invoice_number  = CharField(null=True)   # INV-2026-00001
    refund_eligible_until = DateTimeField(null=True)  # paid_at + 7 days

class GSTInvoice(Model):
    order           = OneToOneField(Order)
    invoice_number  = CharField(unique=True)
    hsn_sac_code    = CharField(default='9992')
    base_amount, cgst, sgst, total_gst, discount, grand_total = DecimalFields
    pdf_url         = URLField(null=True)
```

4.9 SystemConfig — Admin-Editable (apps/admin_panel/models.py)

```
class SystemConfig(Model):  # singleton — always id=1
    product_base_price      = DecimalField(default=200)
    gst_rate                = DecimalField(default=0.1800)
    direct_commission       = DecimalField(default=30)
    upline_commission       = DecimalField(default=10)
    earning_cap             = DecimalField(default=22200)
    sponsor_slot_expiry_days = IntegerField(default=20)
    tds_194h_rate           = DecimalField(default=0.0200)
    tds_194r_rate           = DecimalField(default=0.1000)
    tds_cash_trigger        = DecimalField(default=20000)
    refund_window_days      = IntegerField(default=7)
    updated_by              = ForeignKey(User, null=True)
```

5. Module 1 — Authentication (OTP + JWT)

Registration Flow

1. Client sends phone OR email → POST /api/v1/auth/send-otp/
2. Backend generates 6-digit OTP, stores in OTPRecord (10 min expiry, max 5 attempts), sends via SMS/Email
3. Client submits OTP + name → POST /api/v1/auth/verify-otp-register/
4. Backend verifies OTP, creates User, auto-generates member_id (MLM000001), referral_code (8-char unique), referral_link
5. Returns JWT access + refresh tokens
6. If ?ref=CODE present in request, sets sponsor FK on new user

Login Flow

7. Client sends phone OR email → POST /api/v1/auth/send-otp/ (purpose=LOGIN)
8. Client submits OTP → POST /api/v1/auth/verify-otp-login/
9. Returns JWT tokens

Authentication Endpoints

Method	Endpoint	Description
POST	/api/v1/auth/send-otp/	Send OTP to phone or email
POST	/api/v1/auth/verify-otp-register/	Verify OTP + create account
POST	/api/v1/auth/verify-otp-login/	Verify OTP + return JWT
POST	/api/v1/auth/token/refresh/	Refresh JWT (SimpleJWT)
POST	/api/v1/auth/logout/	Blacklist refresh token
GET	/api/v1/auth/me/	Current user profile
PATCH	/api/v1/auth/me/	Update profile
POST	/api/v1/auth/kyc/submit/	Submit PAN + Aadhaar
GET	/api/v1/auth/kyc/status/	KYC verification status
POST	/api/v1/auth/bank/	Add / update bank or UPI details
POST	/api/v1/admin/auth/login/	Admin email+password login
POST	/api/v1/admin/auth/send-otp/	Admin OTP step 1

Method	Endpoint	Description
POST	/api/v1/admin/auth/verify-otp/	Admin OTP step 2

Implementation Notes

- Use djangorestframework-simplejwt with token blacklisting enabled
- OTP rate-limit: max 3 sends per phone/email per 10 min via Redis counter
- OTP attempt-limit: 5 wrong attempts → invalidate OTPRecord
- referral_code: secrets.token_urlsafe(6).upper() — ensure uniqueness
- member_id: 'MLM' + zero-padded auto-increment → MLM000001
- Admin roles: is_admin=True, role in ['SUPER_ADMIN','FINANCE','SUPPORT']
- Custom DRF permission classes: IsAdminUser, IsFinanceAdmin, IsSupportAdmin

6. Module 2 — Referral Code & Link System

Referral Endpoints

Method	Endpoint	Description
GET	/api/v1/user/referral/	Own referral code, link, and QR URL
GET	/api/v1/user/referral/stats/	Referral count + earnings from referrals
GET	/api/v1/user/referral/list/	Paginated list of direct referrals
POST	/api/v1/auth/validate-referral/	Public: validate a referral code (returns sponsor name only)

Rules

- Referral code is auto-generated at registration, never expires
- sponsor FK is set at registration and is immutable — cannot be changed
- Codes are case-insensitive at validation but stored uppercase
- Membership (tree placement) is only activated after first course purchase

7. Module 3 — Binary Tree Placement Engine

Business Logic

- Every user who completes a course purchase gets a BinaryNode
- Placement: BFS auto-placement — scan left to right, level by level, fill first available LEFT slot, then RIGHT
- One-credit rule: a member earns EITHER ₹30 (direct referrer) OR ₹10 (binary upline) from one joiner — never both
- Sponsor always earns ₹30 and is excluded from the ₹10 upline chain for that joiner

Placement Algorithm (BinaryTreeService)

```
def place_member(new_user, sponsor_user):
    # 1. If sponsor has no left child → place as left child of sponsor's node
    # 2. If sponsor has left but no right → place as right child
```

```
    # 3. Else: BFS from sponsor's node downward, find first empty slot (left priority)
    # 4. Create BinaryNode with:
    #      parent = found parent node
    #      position = LEFT or RIGHT
    #      level = parent.level + 1
    #      upline_l1 = found parent node.user
    #      upline_l2 = found_parent_node.parent.user (if exists)
    #      upline_l3 = found parent node.parent.parent.user (if exists)
    # 5. Link: set parent.left_child or parent.right_child FK
```

Tree Endpoints

Method	Endpoint	Description
GET	/api/v1/user/tree/	Own binary tree (up to 4 levels)
GET	/api/v1/user/tree/uplines/	My upline chain L1–L4
GET	/api/v1/user/tree/downlines/	All downlines (paginated)
GET	/api/v1/user/tree/level/{n}/	All members at tree level N under me
GET	/api/v1/admin/tree/user/{user_id}/	Full tree for any user (admin)
GET	/api/v1/admin/tree/platform/	Platform-wide tree (paginated)

8. Module 4 — Commission Engine

CommissionEngine.process_order() — Step-by-Step

10. Credit ₹30 to sponsor (direct referrer) — always paid
11. Walk binary upline of placement node up to 3 levels; skip any upline that is the same person as the sponsor; credit ₹10 to each eligible upline
12. Check earning cap (₹22,200) for each recipient — credit only up to remaining headroom; mark account CAPPED if cap reached
13. Increment sponsor's direct referral count → check milestone triggers
14. Check if wallet.total_earned crosses a withdrawal band threshold → trigger band action (CASH available or SLOT batch issued)
15. Log all credits in CommissionLedger, update Wallet balances, create WalletTransactions
16. Write to AuditLog
17. Send push notifications to all credited members (async via Celery)

Commission Endpoints

Method	Endpoint	Description
GET	/api/v1/user/commissions/	Full commission ledger (paginated)
GET	/api/v1/user/commissions/summary/	Totals: direct, upline, milestone, tree passive
GET	/api/v1/user/commissions/milestones/	Milestone progress and history
GET	/api/v1/user/commissions/tds/	TDS summary for current FY
GET	/api/v1/admin/commissions/	All commissions (filterable)
GET	/api/v1/admin/commissions/pending/	Pending credits queue
POST	/api/v1/admin/commissions/force-credit/	Manually trigger a credit (Super Admin only)
GET	/api/v1/admin/commissions/tds-report/	Monthly TDS liability report
GET	/api/v1/admin/commissions/export/	CSV / PDF export

9. Module 5 — Wallet & Withdrawal Band System

Withdrawal Flow

18. System tracks wallet.total_earned against the 9-band table
19. CASH band crossed → cash amount becomes available for withdrawal request
20. SLOT band crossed → SponsorSlotService.issue_batch() auto-triggered (5 codes)
21. Member initiates withdrawal request for available CASH band
22. TDS calculated: if FY cash withdrawn > ₹20,000 → apply 2% (194H) or 10% (194R)
23. Admin approves → Razorpay Payout API triggered
24. Payout success webhook → status=PAID, UTR stored, wallet debited

Wallet Endpoints

Method	Endpoint	Description
GET	/api/v1/user/wallet/	Balance, band status, FY summary
GET	/api/v1/user/wallet/transactions/	Transaction history (paginated)
GET	/api/v1/user/wallet/bands/	All 9 bands with status
POST	/api/v1/user/wallet/withdraw/	Request cash withdrawal (specify band)
GET	/api/v1/user/wallet/withdrawals/	Withdrawal history
GET	/api/v1/admin/withdrawals/	All withdrawals (admin)
GET	/api/v1/admin/withdrawals/pending/	Approval queue
POST	/api/v1/admin/withdrawals/{id}/approve/	Approve + trigger Razorpay payout
POST	/api/v1/admin/withdrawals/{id}/reject/	Reject with reason
POST	/api/v1/admin/withdrawals/batch-process/	Batch approve multiple
GET	/api/v1/admin/withdrawals/export/	CSV export for accounting

10. Module 6 — Sponsored Enrollment Slot System

Why Slots Instead of PINs?

Concern	PIN System	Sponsor Slot System
Offline cash exchange	YES — invisible to platform	NO — all transactions on-platform
GST invoice for every enrollment	NO — tax evasion risk	YES — full ₹241.72 GST invoice
Full audit trail	PARTIAL	COMPLETE — all events timestamped
RBI monetary instrument risk	HIGH — PSS Act 2007	NONE — code has no cash value
Double earning by sponsor	YES — ₹230 from ₹200 sale	NO — only ₹30 on-platform

How It Works — Step by Step

25. Member's earnings cross a SLOT band → system auto-generates 5 unique SPONSOR-XXXX codes with 20-day expiry
26. Member shares code via platform's WhatsApp / SMS / Email / Copy button — share event recorded
27. New joiner enters code at checkout → cart ₹241.72 − ₹241.72 = ₹0 due → full GST invoice still generated
28. New joiner gets course access; sponsor gets ₹30 direct commission; binary uplines L2–L4 get ₹10 each
29. If code unused within 20 days → auto-expired (Day 7 and Day 2 SMS warnings sent via Celery)

Sponsor Slot Endpoints

Method	Endpoint	Description
GET	/api/v1/user/sponsor-slots/	My slot batches + all codes with status
POST	/api/v1/user/sponsor-slots/{code}/share/	Log share event (WhatsApp/SMS/Email/Copy)
POST	/api/v1/sponsor-slots/validate/	Public: validate a slot code at checkout
GET	/api/v1/admin/sponsor-slots/	All batches platform-wide
GET	/api/v1/admin/sponsor-slots/flagged/	Abuse-flagged codes
POST	/api/v1/admin/sponsor-slots/{code}/expire/	Manually expire a code

11. Module 7 — Course / E-Book Management

Purchase Flow

30. User visits course page → GET /api/v1/courses/ — sees active e-books
31. User initiates purchase → POST /api/v1/payments/create-order/
    - If sponsor_code provided: validate via SponsorSlotService.validate_code()
    - Valid code: effective amount = ₹0 (full discount), GST invoice still generated showing full ₹241.72
    - No code: full ₹241.72 via Razorpay
32. Razorpay order created → client completes payment in Razorpay SDK
33. Payment success → POST /api/v1/payments/verify/ — HMAC signature verified
34. Order.status=PAID, Enrollment created, e-book access granted
35. CommissionEngine.process_order() called asynchronously via Celery
36. GST invoice PDF generated async via Celery (reportlab / weasyprint), stored on Azure Blob

Course Endpoints

Method	Endpoint	Description
GET	/api/v1/courses/	List all active e-books (public)
GET	/api/v1/courses/{slug}/	E-book detail (public)
GET	/api/v1/user/courses/enrolled/	My enrolled e-books
GET	/api/v1/user/courses/{slug}/download/	Signed download URL (15-min expiry)
GET	/api/v1/admin/courses/	All e-books (admin)
POST	/api/v1/admin/courses/	Create new e-book
PATCH	/api/v1/admin/courses/{id}/	Update e-book
DELETE	/api/v1/admin/courses/{id}/	Soft delete (is_active=False)
POST	/api/v1/admin/courses/{id}/upload/	Upload PDF to Azure Blob
GET	/api/v1/admin/courses/enrollments/	All enrollments (filterable)

12. Module 8 — Payment & Payout Management

Razorpay Checkout Integration

37. POST /api/v1/payments/create-order/ → call Razorpay Orders API, return {razorpay_order_id, key_id, amount}
38. Client pays via Razorpay SDK (UPI / Card / Netbanking)
39. Client posts {payment_id, order_id, signature} to POST /api/v1/payments/verify/
40. Backend verifies HMAC: razorpay.utility.verify_payment_signature()
41. On success: mark PAID → enrollment + commission pipeline triggered

Refund (7-day window)

42. POST /api/v1/user/orders/{id}/refund/ — user requests refund
43. Validate: paid + within 7-day window + not already refunded
44. Call Razorpay Refunds API
45. On success: CommissionEngine.reverse_commissions(order) — all recipients debited
46. Enrollment access revoked, Order.status=REFUNDED

Payment Endpoints

Method	Endpoint	Description
POST	/api/v1/payments/create-order/	Create Razorpay order
POST	/api/v1/payments/verify/	Verify signature + complete purchase
POST	/api/v1/payments/webhook/	Razorpay webhook (payment + payout events)
GET	/api/v1/user/orders/	My purchase history
GET	/api/v1/user/orders/{id}/invoice/	Download GST invoice PDF
POST	/api/v1/user/orders/{id}/refund/	Request refund (within 7 days)
GET	/api/v1/admin/orders/	All orders (filterable + exportable)
GET	/api/v1/admin/revenue/	Revenue dashboard (daily/weekly/monthly)
GET	/api/v1/admin/gst-report/	Monthly GST report (GSTR-1 data)

13. Module 9 — Admin Panel APIs

Role-Based Access Control

Role	Access Level
Super Admin	Full access to all modules + system configuration
Finance	Withdrawals, payouts, TDS/GST reports, commission ledger
Support	User management, KYC queue, grievances (read-only on financials)

Admin Dashboard KPIs — GET /api/v1/admin/dashboard/

- Total members | New today / week / month

- Total revenue | Today / week / month
- Pending withdrawals count + total amount
- Pending KYC verifications count
- Active sponsor slot codes count
- Flagged abuse cases
- Retail vs MLM purchase ratio (target >30% retail — DS Rules 2021)

Admin Endpoints

Method	Endpoint	Description
GET	/api/v1/admin/users/	List all users (paginated, filterable)
PATCH	/api/v1/admin/users/{id}/	Edit user (role, status, KYC)
POST	/api/v1/admin/users/{id}/suspend/	Suspend with reason
GET	/api/v1/admin/users/kyc-queue/	Pending KYC verifications
POST	/api/v1/admin/users/{id}/kyc/verify/	Approve KYC
GET	/api/v1/admin/users/delisted/	De-listed member registry
GET	/api/v1/admin/config/	Current SystemConfig values
PATCH	/api/v1/admin/config/	Update config (Super Admin only)
GET	/api/v1/admin/reports/tds/	TDS deducted this month/quarter/FY
GET	/api/v1/admin/reports/gst/	GST collected + GSTR-1 filing data
GET	/api/v1/admin/reports/retail-ratio/	Retail vs MLM ratio (compliance)
GET	/api/v1/admin/reports/compliance/	DoCA filing-ready report
GET	/api/v1/admin/grievances/	Grievance redressal queue
POST	/api/v1/admin/grievances/{id}/respond/	Respond + update status

14. Celery Background Tasks

Task	Trigger	Schedule
process_commission_task(order_id)	After Razorpay payment verified	Immediate async
generate_gst_invoice_pdf(order_id)	After order marked PAID	Immediate async
expire_sponsor_slots()	Daily cleanup	Midnight IST (Celery Beat)
notify_slot_expiry_day7(code_id)	7 days before slot expiry	Celery ETA
notify_slot_expiry_day2(code_id)	2 days before slot expiry	Celery ETA
send_otp_sms / send_otp_email	On OTP request	Immediate async
notify_commission_credited	After commission credit	Immediate async
notify_milestone_reached	After milestone trigger	Immediate async
poll_payout_status()	Monitor PROCESSING payouts	Every 5 min (Beat)
check_earning_cap(user_id)	After every commission credit	Immediate async

15. Security Requirements

- JWT via djangorestframework-simplejwt with token blacklisting
- OTP rate-limit: 3 sends / 10 min per phone/email (Redis counter)
- Login lockout: 5 wrong OTPs → 30-min account lock
- API throttle: 100 requests/min per authenticated user
- CORS: whitelist only Flutter and Next.js origins
- Razorpay webhook: always verify HMAC signature before processing
- Signed download URLs: 15-min expiry via Azure SAS tokens
- Idempotency: payment verify endpoint checks Order.status — no double commission credits
- Aadhaar stored masked: XXXX-XXXX-1234 format
- Admin 2FA enforced for all admin accounts

16. API Conventions

Standard Response Envelope

```
// Success
{ "success": true, "data": { ... }, "message": "Operation successful", "errors": null }

// Error
{ "success": false, "data": null, "message": "Validation failed",
  "errors": { "field": ["error message"] } }

// Pagination
{ "count": 100, "next": "...", "previous": "...", "results": [...] }
```

- All endpoints versioned: /api/v1/
- Monetary amounts returned as strings with 2 decimal places: "amount": "30.00"
- All timestamps in ISO 8601 format, IST timezone
- Authentication header: Authorization: Bearer <access_token>

17. Critical Business Rules — Implement as Validations

#	Rule	Detail
1	One-credit rule	Never credit both ₹30 and ₹10 to same person for same order
2	Earning cap	No member earns more than ₹22,200 total; credits capped at remaining headroom
3	Slot ≠ cash	₹1,000 slot band values are NOT cash — issued as 5 discount codes only
4	GST on ₹0 orders	Every order including ₹0 sponsor-slot orders must generate a GST invoice
5	Commission reversal on refund	All commissions reversed from all recipients if refund within 7 days
6	TDS trigger	2% (194H) on commission cash once FY total > ₹20,000; 10% (194R) on milestone bonuses
7	PAN mandatory	No PAN at withdrawal = 20% TDS
8	No self-redemption	Sponsor slot code cannot be redeemed by its own issuer
9	Membership after purchase	User registered but not yet purchased = NOT in tree, referral link inactive
10	Retail purchase	Non-member buying at ₹241.72 gets course access but generates no MLM commissions
11	Rejoin	After earning cap hit, member re-purchases → new member_id, zero earnings, existing tree position retained

18. Technology Stack & Requirements

Layer	Technology
Backend Framework	Python 3.12 · Django 5.x · Django REST Framework 3.15.x
Authentication	djangorestframework-simplejwt 5.3.x (token blacklisting)
Database	MySQL 8.x via mysqlclient · Django ORM
Cache / Broker	Redis 7.x
Async Tasks	Celery 5.4.x + Celery Beat
Payment Gateway	razorpay 1.4.x (checkout + payouts + refunds)
PDF Generation	reportlab 4.x or weasyprint 62.x
Cloud Storage	Azure Blob Storage (e-book files + invoice PDFs)
Web Server	Gunicorn 22.x + Nginx
Containerization	Docker + Docker Compose
CI/CD	GitHub Actions
Testing	pytest-django 4.8.x · factory-boy 3.3.x · responses (Razorpay mock)
Security / Rate Limiting	django-axes or DRF throttle · django-cors-headers

19. Recommended Build Order for Cursor

Phase	Days	Deliverable
Phase 1	Days 1–3	Docker setup · User model · OTP Auth · JWT · basic User CRUD
Phase 2	Days 4–6	Binary Tree model + BFS placement service · Referral code generation
Phase 3	Days 7–9	Razorpay checkout · Order model · GST invoice engine · Enrollment
Phase 4	Days 10–12	Commission engine · Wallet · Milestone bonuses · Earning cap
Phase 5	Days 13–15	Sponsor Slot System · Withdrawal bands · TDS calculation
Phase 6	Days 16–18	Admin panel APIs · reporting endpoints · SystemConfig
Phase 7	Days 19–20	Celery tasks · Notifications · Audit logging · Security hardening
Phase 8	Days 21+	Testing · Razorpay payout integration · Compliance reports

For each module: write models → migrations → serializers → services (business logic) → views → URLs → tests.
Always keep business logic in services.py — not in views or models.
Use factory_boy for test fixtures. Mock all Razorpay API calls with the responses library.